CellManager: 셀 관리 최적화

개요

CellManager는 Halo2 회로에서 셀(cell) 할당을 효율적으로 관리하는 핵심 컴포넌트입니다. ZKEVM 구현체에서는 두 가지 서로 다른 CellManager 전략을 사용합니다:

1. **EVM Circuit의 CellManager**: 수평 정렬(horizontal alignment) 전략
2. **Keccak Circuit의 CellManager**: 수직 정렬(vertical alignment) 전략

각 전략은 해당 회로의 특성에 맞게 최적화되어 있으며, 셀 사용을 최소화하고 회로 크기를 줄이는 데 중요한 역할을 합니다.

EVM Circuit의 CellManager (수평 정렬)

구조

EVM Circuit의 CellManager는 **수평 우선(horizontal-first)** 할당 전략을 사용합니다. 이는 각 컬럼을 독립적으로 관리하며, 셀을 요청할 때 가장 높이가 낮은 컬럼을 선택하여 할당합니다.

파일: `zkevm-circuits/src/evm_circuit/util.rs`

```
#[derive(Clone, Debug)]

pub(crate) struct CellManager<F> {

width: usize, // 컬럼의 개수

height: usize, // 최대 높이

cells: Vec<Cell<F>>, // 미리 쿼리된 모든 셀

columns: Vec<CellColumn<F>>, // 컬럼 메타데이터
```

```
}
```

초기화 과정

CellManager는 생성 시 모든 셀을 미리 쿼리합니다:

```
pub(crate) fn new(
meta: &mut ConstraintSystem<F>,
height: usize,
advices: &[Column<Advice>],
height_offset: usize,
) -> Self {
let width = advices.len();
let mut cells = Vec::with_capacity(height * width);
let mut columns = Vec::with_capacity(width);
// 모든 셀을 미리 쿼리
query_expression(meta, |meta| {
for c in 0..width {
for r in 0..height {
cells.push(Cell::new(meta, advices[c], height_offset + r, c));
}
columns.push(CellColumn {
index: c,
cell_type: CellType::StoragePhase1,
```

```
height: 0,
expr: cells[c * height].expr(),
});
}
});
// ...
}
```

컬럼 타입 분류

EVM Circuit은 컬럼을 용도에 따라 분류합니다:

1. **Lookup 컬럼**: 특정 테이블 룩업 전용 (`CellType::Lookup(table)`)
2. **Phase2 Copy 컬럼**: Phase 2에서 복사 제약 조건 사용 (`CellType::StoragePermutationPhase2`)
3. **Phase2 Storage 컬럼**: Phase 2 일반 저장소 (`CellType::StoragePhase2`)
4. **Copy 컬럼**: Phase 1에서 복사 제약 조건 사용 (`CellType::StoragePermutation`)
5. **Byte Lookup 컬럼**: 바이트 룩업 전용 (`CellType::LookupByte`)
6. **일반 Storage 컬럼**: Phase 1 일반 저장소 (`CellType::StoragePhase1`)

```
// Lookup 컬럼 마킹
for &(table, count) in LOOKUP_CONFIG {
for _ in 0usize..count {
columns[column_idx].cell_type = CellType::Lookup(table);
column_idx += 1;
```

```
}
}

// Phase2 Copy 컬럼 마킹
for _ in 0..N_PHASE2_COPY_COLUMNS {
meta.enable_equality(advices[column_idx]);
columns[column_idx].cell_type = CellType::StoragePermutationPhase2;
column_idx += 1;
}
```

셀 할당 전략

셀을 요청할 때, CellManager는 다음 알고리즘을 사용합니다:

1. 요청된 `CellType` 과 일치하는 컬럼 중에서 **가장 높이가 낮은 컬럼**을 선택
2. 해당 컬럼의 다음 사용 가능한 행에 셀 할당
3. 컬럼의 높이 카운터 증가

```
pub(crate) fn query_cell(&mut self, cell_type: CellType) -> Cell<F> {
self.query_cells(cell_type, 1)[0].clone()
}

pub(crate) fn query_cells(&mut self, cell_type: CellType, count: usize) -> Vec
let mut cells = Vec::with_capacity(count);
while cells.len() < count {
```

```
let column_idx = self.next_column(cell_type);
let column = &mut self.columns[column_idx];
cells.push(self.cells[column_idx * self.height + column.height].clone());
column.height += 1;
}
cells
}


fn next_column(&self, cell_type: CellType) -> usize {
let mut best_index: Option<usize> = None;
let mut best_height = self.height;
// 가장 높이가 낮은 컬럼 찾기
for column in self.columns.iter() {
if column.cell_type == cell_type && column.height < best_height {
best_index = Some(column.index);
best_height = column.height;
}
}
// StoragePermutation 컬럼을 Storage 대신 사용 가능
if cell_type == CellType::StoragePhase1 {
for column in self.columns.iter() {
if column.cell_type == CellType::StoragePermutation
&& column.height < best_height {
```

```
best_index = Some(column.index);

best_height = column.height;

}

}

}

best_index.expect("not enough cells for query")

}
```

최적화 효과

이 수평 정렬 전략의 장점:

1. **균등한 컬럼 사용**: 모든 컬럼의 높이를 비슷하게 유지하여 회로 크기 최소화
2. **타입별 분리**: 각 컬럼의 용도를 명확히 하여 제약 조건 구성 단순화
3. **미리 쿼리**: 모든 셀을 초기화 시 쿼리하여 런타임 오버헤드 제거
4. **유연한 대체**: Storage 타입 요청 시 Permutation 컬럼도 사용 가능

Step에서의 사용 예시

파일: `zkevm-circuits/src/evm_circuit/step.rs`

```
pub(crate) struct Step<F> {

pub(crate) state: StepState<F>,

pub(crate) cell_manager: CellManager<F>,

}
```

```
impl<F: Field> Step<F> {
pub(crate) fn  new(
meta: &mut ConstraintSystem<F>,
advices: [Column<Advice>; STEP_WIDTH],
offset: usize,
is_next: bool,
) -> Self {
let  height = if  is_next {
STEP_STATE_HEIGHT
} else {
MAX_STEP_HEIGHT
};
let  mut  cell_manager = CellManager::new(meta, height, &advices, offset);
// StepState 필드들을 순차적으로 할당
let  state = StepState {
execution_state: DynamicSelectorHalf::new(
&mut  cell_manager,
ExecutionState::amount(),
),
rw_counter: cell_manager.query_cell(CellType::StoragePhase1),
call_id: cell_manager.query_cell(CellType::StoragePhase1),
tx_id: cell_manager.query_cell(CellType::StoragePhase1),
// ... 더 많은 필드들
```

```
};
Self { state, cell_manager }
}
}
```

통계 수집

CellManager는 사용 통계를 제공합니다:

```
pub(crate) fn get_stats(&self) -> BTreeMap<CellType, (usize, usize, usize)> {
let mut data = BTreeMap::new();
for column in self.columns.iter() {
let (mut count, mut height, mut num_cells) =
data.get(&column.cell_type).unwrap_or(&(0, 0, 0));
count += 1; // 컬럼 개수
height = height.max(column.height); // 최대 높이
num_cells += column.height; // 총 셀 개수
data.insert(column.cell_type, (count, height, num_cells));
}
data
}
```

이를 통해 각 CellType별로:

- 사용된 컬럼 개수
- 최대 높이

- 총 사용된 셀 개수

를 추적할 수 있습니다.

Keccak Circuit의 CellManager (수직 정렬)

구조

Keccak Circuit의 CellManager는 **수직 우선(vertical-first)** 할당 전략을 사용합니다. 이는 행(row)을 우선적으로 채우고, 필요에 따라 새로운 컬럼을 동적으로 추가합니다.

파일: `zkevm-circuits/src/keccak_circuit/cell_manager.rs`

```
#[derive(Clone, Debug)]
pub struct CellManager<F> {
height: usize, // 회로의 높이
columns: Vec<CellColumn<F>>, // 동적으로 추가되는 컬럼들
rows: Vec<usize>, // 각 행의 현재 컬럼 위치
num_unused_cells: usize, // 사용되지 않은 셀 개수
}
```

초기화

Keccak Circuit의 CellManager는 빈 상태로 시작하여 필요에 따라 컬럼을 추가합니다:

```
pub(crate) fn new(height: usize) -> Self {
Self {
height,
```

```
columns: Vec::new(),
rows: vec![0; height], // 각 행은 컬럼 0부터 시작
num_unused_cells: 0,
}
}
```

셀 할당 전략

Keccak Circuit은 두 가지 셀 할당 방식을 제공합니다:

1. 자동 위치 선택 (`query_cell`)

가장 사용이 적은 행을 찾아 셀을 할당합니다:

```
pub(crate) fn query_cell(&mut self, meta: &mut ConstraintSystem<F>) -> Cell<F>
let (row_idx, column_idx) = self.get_position();
self.query_cell_at_pos(meta, row_idx as i32, column_idx)
}

fn get_position(&mut self) -> (usize, usize) {
let mut best_row_idx = 0usize;
let mut best_row_pos = 100000usize;
// 가장 적게 사용된 행 찾기
for (row_idx, row) in self.rows.iter().enumerate() {
if *row < best_row_pos {
best_row_pos = *row;
```

```
best_row_idx = row_idx;
}
}
self.rows[best_row_idx] += 1;
(best_row_idx, best_row_pos)
}
```

2. 특정 행 지정 (`query_cell_at_row`)

특정 행에 셀을 할당합니다:

```
pub(crate) fn query_cell_at_row(
&mut self,
meta: &mut ConstraintSystem<F>,
row_idx: i32,
) -> Cell<F> {
let column_idx = self.rows[row_idx as usize];
self.rows[row_idx as usize] += 1;
self.query_cell_at_pos(meta, row_idx, column_idx)
}
```

동적 컬럼 생성

필요한 경우 새로운 컬럼을 동적으로 생성합니다:

```
fn query_cell_at_pos(
&mut self,
meta: &mut ConstraintSystem<F>,
row_idx: i32,
column_idx: usize,
) -> Cell<F> {
let column = if column_idx < self.columns.len() {
// 기존 컬럼 사용
self.columns[column_idx].advice
} else {
// 새 컬럼 생성
let advice = meta.advice_column();
let mut expr = 0.expr();
meta.create_gate("Query column", |meta| {
expr = meta.query_advice(advice, Rotation::cur());
vec![0.expr()]
});
self.columns.push(CellColumn { advice, expr });
advice
};


let mut cells = Vec::new();
```

```
meta.create_gate("Query cell", |meta| {
cells.push(Cell::new(meta, column, column_idx, row_idx));
vec![0.expr()]
});
cells[0].clone()
}
```

Region 정렬

Region을 시작할 때 모든 행을 같은 컬럼 위치로 정렬합니다:

```
pub(crate) fn start_region(&mut self) -> usize {
// 모든 행이 같은 컬럼에서 시작하도록 정렬
let width = self.get_width();
for row in self.rows.iter_mut() {
self.num_unused_cells += width - *row;
*row = width;
}
width
}

pub(crate) fn get_width(&self) -> usize {
self.rows.iter().cloned().max().unwrap()
}
```

이는 각 Keccak 블록이 정렬된 컬럼에서 시작하도록 보장합니다.

Cell 구조

Keccak Circuit의 Cell은 표현식과 할당 정보를 모두 포함합니다:

```
#[derive(Clone, Debug)]

pub(crate) struct Cell<F> {

pub(crate) expression: Expression<F>, // 제약 조건용 표현식

pub(crate) column_expression: Expression<F>, // 컬럼 표현식

pub(crate) column: Option<Column<Advice>>, // 실제 컬럼

pub(crate) column_idx: usize, // 컬럼 인덱스

pub(crate) rotation: i32, // 행 오프셋

}


impl<F: Field> Cell<F> {

pub(crate) fn  assign(&self, region: &mut KeccakRegion<F>, offset: i32, value: F

region.assign(

self.column_idx,

(offset + self.rotation) as usize,

value

);

}

pub(crate) fn  at_offset(&self, meta: &mut ConstraintSystem<F>, offset: i32) ->

// 다른 오프셋에서 같은 컬럼의 셀 쿼리
```

```
// ...
}
}
```

최적화 효과

수직 정렬 전략의 장점:

1. **밀집된 배치**: 행을 우선적으로 채워 셀 사용 밀도 최대화
2. **동적 확장**: 필요한 만큼만 컬럼 생성하여 메모리 효율성 향상
3. **Region 정렬**: 각 Keccak 블록이 정렬된 위치에서 시작하여 구조 명확화
4. **유연한 할당**: 자동 또는 수동 행 선택 지원

Keccak Circuit에서의 사용

Keccak Circuit은 여러 라운드에 걸쳐 셀을 할당합니다:

```
// 각 라운드마다 필요한 셀 할당
for round in 0..NUM_ROUNDS {
for i in 0..5 {
for j in 0..5 {
// 특정 행에 셀 할당
let cell = cell_manager.query_cell_at_row(meta, row_idx);
// ...
}
}
```

```
}


// Region 시작 전 정렬
let start_column = cell_manager.start_region();
```

두 전략의 비교

특성	EVM Circuit (수평)	Keccak Circuit (수직)
할당 방향	컬럼 우선 (가로)	행 우선 (세로)
초기화	모든 셀 미리 쿼리	빈 상태로 시작
컬럼 관리	고정된 컬럼 세트	동적 컬럼 생성
타입 분류	명시적 CellType	타입 구분 없음
최적화 목표	컬럼 간 균등 분배	행 밀집도 최대화
사용 사례	복잡한 상태 머신	반복적 해시 연산

셀 할당 최적화 기법

1. 미리 쿼리 (Pre-querying)

EVM Circuit은 모든 셀을 초기화 시 쿼리하여 런타임 오버헤드를 제거합니다:

```
// 초기화 시 모든 셀 쿼리
for c in 0..width {
for r in 0..height {
```

```
cells.push(Cell::new(meta, advices[c], height_offset + r, c));
}
}
```

장점:

- 제약 조건 구성 시 빠른 셀 접근
- 메모리 지역성 향상
- 예측 가능한 메모리 사용

2. 지연 할당 (Lazy Allocation)

Keccak Circuit은 필요할 때만 컬럼을 생성합니다:

```
let column = if column_idx < self.columns.len() {
self.columns[column_idx].advice
} else {
let advice = meta.advice_column();
// 새 컬럼 생성 및 추가
self.columns.push(CellColumn { advice, expr });
advice
};
```

장점:

- 메모리 효율성
- 실제 사용량에 맞는 회로 크기
- 유연한 확장성

3. 타입 기반 분리

EVM Circuit은 셀을 용도별로 분류하여 관리합니다:

```
pub enum CellType {
StoragePhase1,
StoragePhase2,
StoragePermutation,
StoragePermutationPhase2,
Lookup(Table),
LookupByte,
}
```

장점:

- 명확한 셀 용도 구분
- 최적화된 제약 조건 구성
- 디버깅 용이성

4. 균등 분배 알고리즘

두 CellManager 모두 사용량이 적은 위치를 우선 선택합니다:

```
// EVM: 가장 높이가 낮은 컬럼 선택
for column in self.columns.iter() {
if column.cell_type == cell_type && column.height < best_height {
best_index = Some(column.index);
```

```
best_height = column.height;
}
}

// Keccak: 가장 사용이 적은 행 선택
for (row_idx, row) in  self.rows.iter().enumerate() {
if *row < best_row_pos {
best_row_pos = *row;
best_row_idx = row_idx;
}
}
```

장점:

- 회로 크기 최소화
- 균등한 리소스 사용
- 핫스팟 방지

성능 고려사항

메모리 사용

- **EVM Circuit**: `width × height` 개의 셀을 미리 할당
- 장점: 빠른 접근
- 단점: 높은 초기 메모리 사용
- **Keccak Circuit**: 필요한 만큼만 할당

- 장점: 낮은 메모리 사용
- 단점: 동적 할당 오버헤드

회로 크기

두 전략 모두 회로 크기를 최소화하는 것을 목표로 합니다:

```
// EVM: 최대 높이 추적
pub(crate) fn get_height(&self) -> usize {
self.columns.iter().map(|column| column.height).max().unwrap()
}

// Keccak: 최대 너비 추적
pub(crate) fn get_width(&self) -> usize {
self.rows.iter().cloned().max().unwrap()
}
```

사용되지 않은 셀 추적

Keccak Circuit은 낭비되는 셀을 추적합니다:

```
pub(crate) fn start_region(&mut self) -> usize {
let width = self.get_width();
for row in self.rows.iter_mut() {
self.num_unused_cells += width - *row; // 낭비된 셀 계산
*row = width;
```

```
}

width

}
```

이를 통해 최적화 기회를 식별할 수 있습니다.

실전 사용 패턴

ExecutionGadget에서의 사용

```
impl<F: Field> ExecutionGadget<F> for SomeGadget<F> {

fn  configure(cb: &mut ConstraintBuilder<F>) -> Self {

// CellManager를 통해 필요한 셀 할당

let  value = cb.query_cell();

let  address = cb.query_word_rlc();

// 제약 조건 구성

cb.require_equal("constraint", value.expr(), expected.expr());

Self { value, address }

}

}
```

Keccak 라운드에서의 사용

```
// 각 라운드마다 상태 셀 할당
```

```
for round in 0..NUM_ROUNDS {
let state_cells: Vec<Vec<Cell<F>>> = (0..5)
.map(|_| {
(0..5)
.map(|_| cell_manager.query_cell_at_row(meta, row_idx))
.collect()
})
.collect();
// 라운드 제약 조건 구성
// ...
}
```

디버깅 및 모니터링

통계 출력

```
// EVM Circuit 통계
let stats = cell_manager.get_stats();
for (cell_type, (count, height, num_cells)) in stats {
println!("{:?}: {} columns, max height {}, {} cells",
cell_type, count, height, num_cells);
}
```

```
// Keccak Circuit 통계

println!("Width: {}", cell_manager.get_width());

println!("Unused cells: {}", cell_manager.get_num_unused_cells());
```

높이 추적

```
// EVM: 각 컬럼의 높이

let  heights = cell_manager.get_heights();

println!("Column heights: {:?}", heights);


// Keccak: 각 행의 사용량

println!("Row usage: {:?}", cell_manager.rows);
```

결론

CellManager는 ZKEVM 회로의 효율성을 결정하는 핵심 컴포넌트입니다:

- **EVM Circuit**은 복잡한 상태 머신에 적합한 수평 정렬 전략을 사용하여 타입별로 셀을 관리하고 균등하게 분배합니다.
- **Keccak Circuit**은 반복적인 해시 연산에 적합한 수직 정렬 전략을 사용하여 밀집된 배치와 동적 확장을 제공합니다.

두 전략 모두 회로 크기를 최소화하고 증명 생성 시간을 단축하는 데 기여하며, 각 회로의 특성에 맞게 최적화되어 있습니다.

관련 문서

- 02-핵심-회로/01-EVM-Circuit.md - EVM Circuit의 전체 구조
- 02-핵심-회로/04-Keccak-Circuit.md - Keccak Circuit의 상세 구현
- 09-구현-세부사항/02-제약-조건-구축.md - ConstraintBuilder와의 통합